

P183 Web Store

Introduction

Au cours de ce projet, il nous a été demandé de rendre une application *Express* plus sécurisée. En effet, nous avons reçu une application web, créée avec *ExpressJS*, pleine de failles. Notre but est de rendre cette application résistante à toutes sortes d'attaques.

Tâches réalisées

1 Implémenter la page de login frontend

J'ai commencé par ajouter les options de navigation sur toutes les pages, ces dernières étaient en commentaire alors ce fut rapide. J'ai ensuite créé un formulaire sur la page de login afin de recevoir les infos utilisateur.

Sur la page de login, j'ai ensuite ajouté du *javascript* afin d'empêcher le formulaire de s'envoyer et à la place construire une requête *POST*. Enfin, nous prenons la requête et nous faisons un *fetch* sur */api/auth/login*.

2 Implémenter une page d'inscription en frontend

De même que pour l'étape précédente, nous commençons par créer le formulaire html sur la page *register.html*. Nous ajoutons ensuite du *JavaScript* afin de gérer la soumission du formulaire à l'adresse */api/auth/register*.

Enfin nous allons dans *AuthController* et nous implémentons la création d'un nouvel utilisateur.

3 Remplacer les mots de passes en clair dans la base par un hash

Afin de pouvoir démarrer cette étape, nous avons besoin d'installer [Argon2](#). Ceci servira à hasher et vérifier les passwords en db. Voici la commande relative à l'installation : `npm install argon2`. À présent, nous pouvons commencer à coder, nous allons dans *AuthController* et, dans la méthode *register*, nous ajoutons le bout de code qui permet de hash le mot de passe :

```
const hashPassword = await argon2.hash(password, {
  salt: Buffer.from("saltThatIsLongEnough"),
});
```

Attention à bien remplacer *password* par *hashPassword* dans la requête SQL.

Une fois fait, nous recréons tous les comptes utilisateurs afin de stocker les mots de passes nouvellement hashés.

4 Ajouter un sel

Lors de cette étape, nous allons générer un sel (une chaîne de caractères aléatoire) pour chaque utilisateur et nous allons l'ajouter au hash du mot de passe. En réalité, le sel est généré par *argon2*

automatiquement. Nous n'avons qu'à retirer la partie où l'on spécifie le sel. Notre code de l'étape précédente devient donc.

```
const hashPassword = await argon2.hash(password);
```

Ceci est à faire pour le login autant que pour le register, sinon les mots de passes ne correspondront pas.

5 Ajouter un poivre

Nous allons maintenant ajouter le poivre à notre application. Le poivre est une (très) longue chaîne de caractères qui est propre à l'application. Le poivre est généralement stocké dans un fichier `.env`, mais jamais dans l'application elle-même. Ce dernier nous permet de renforcer les mots de passes des utilisateurs en ajoutant de la complexité. Dans le fichier `.env` à la racine du projet, nous ajoutons donc, par exemple, `PEPPER=9f3c2a8e7b1d4c6f8a91e2b5c7d9f0a1bd55672d7ecdef0ad6c46739ebcaef0`. Sentez-vous libre de changer la valeur du poivre.

Ensuite nous ajoutons le poivre au mot de passe ainsi (toujours les mêmes parties du code):

```
//ajouter le poivre au pwd
const pepper = process.env.PEPPER;
let passwordWithPepper = password + pepper;
//hash le pwd
const hashPassword = await argon2.hash(passwordWithPepper);
```

6 Prévenir les injections SQL

Il est maintenant temps de revoir nos requêtes SQL et de les sécuriser afin de rendre les injections SQL impossibles. Nous n'avons qu'à substituer les valeurs dans la requête par des points d'interrogation. Nous définissons ensuite le contenu des ? lors de l'appel de la méthode `db.query`.

Voici une proposition afin de refactoriser le code. Créer une méthode afin de récupérer les infos d'un utilisateur à l'aide de son email.

```
async function getUserInfos(email) {
  const query = `SELECT password, username, role, id FROM users WHERE email
= ? LIMIT 1`;
  const results = await new Promise((resolve, reject) => {
    db.query(query, [email], (err, results) => {
      if (err) reject(err);
      else resolve(results);
    });
  });

  return results[0];
}
```

Vous pouvez remarquer qu'ici nous avons employé une *promise*. En effet, cela nous facilite la gestion des erreurs de la requête, nous pouvons maintenant l'utiliser ainsi.

```
const result = await getUserInfos(email).catch((err) => {
  res.status(500).json({ error: "Quelque chose s'est mal passé" });
  return;
});
```

7 Implémenter l'utilisation d'un token jwt

À présent, nous allons restreindre l'accès à l'application afin de la rendre plus safe. Nous allons utiliser des JSON Web Tokens. Pour ceci, créez une nouvelle variable d'environnement, puis rendez-vous dans le AuthController, dans la partie login. La variable d'environnement s'appelle *JWT_SECRET* et contient une suite hexadécimale complexe. Pour en créer une facilement, vous pouvez visiter [ce site](#).

Nous créons un token au moment où l'utilisateur se login, un utilisateur qui vient de se créer un compte ne sera donc pas authentifié. Le token est stocké dans un cookie afin d'empêcher l'utilisateur d'interagir avec.

Il nous faut maintenant protéger les routes et pour ceci il nous faut de quoi tester et valider le token. Nous allons utiliser le middleware d'authentification (le fichier */middleware/auth.js*). Nous y ajoutons et exportons la méthode *verifyToken*. Cette dernière prend le token des cookies et vérifie son contenu à l'aide de la variable d'environnement.

Il ne nous reste plus qu'à utiliser la méthode *verifyToken* pour toutes les routes qui en ont besoin. Il suffit d'ajouter le nom de la méthode de cette manière. *app.get("/profile", (_req, res) => ... -> app.get("/profile", verifyToken, (_req, res) => ...*

8 Ajouter les rôles administrateur et utilisateur dans le jwt et protéger les routes d'administration

Puisque nous avons déjà placé le rôle utilisateur dans le token, il nous suffit de checker le rôle utilisateur lors de requêtes vers les routes admin (*/admin* et */api/admin*). Nous créons donc une méthode dans *auth.js* (du middleware) qui permet de checker le rôle que nous utilisons ensuite de la même manière que la méthode de vérification du token.

```
//permet de check le rôle
function verifyAdmin(req, res, next) {
  if (req.user.role !== "admin") {
    return res
      .status(403)
      .json({ message: "Accès refusé : rôle administrateur requis" });
  } else if (req.user.role === "admin") {
    //else if pour être certain que le rôle est bien admin avant de faire
    next();
  }
}
```

9 Implémenter le https

Il est maintenant temps de passer à la connection https. L'utilisation du protocole de connection https requiert un certificat et une clé privée du côté du serveur. Nous allons utiliser OpenSSL afin de créer tout ça. Pour ce faire, créez un dossier *certs* dans le dossier *app* et ouvrez un terminal dedans. Vous pouvez ensuite lancer cette commande qui va créer tout ce dont nous avons besoin. `openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 365 -subj "/CN=localhost" -addext "subjectAltName=DNS:localhost,IP:127.0.0.1"`

Étape suivante : gérer le fichier .env : trouvez le fichier .env et ne laissez dedans que `COMPOSE_PROJECT_NAME=webshop_183`. Coupez le reste et collez-le dans un nouveau fichier .env dans le dossier *app*. Si vous avez perdu le contenu de l'ancien .env, vous pouvez renommer le `/app/.env.example` en .env

À présent nous devons modifier les import des autres ressources dans les fichiers *db.js* et *app.js*.

```
//on passe de ça
const express = require("express");
//à ça ->
import express from "express";
```

Une fois fait, nous passons le serveur en mode https. Il sera nécessaire d'installer le package https `npm i https`. Nous l'utilisons dans le fichier *server.js*.

```
// Démarrage du serveur
https.createServer(options, app).listen(8080, () => {
  console.log("Serveur démarré sur https://localhost:8080");
});
```

10 politique de mot de passe

Nous nous attaquons maintenant à la politique du mot de passe. Le but est d'empêcher les utilisateurs de créer un compte avec un mot de passe trop faible. La politique que j'ai implémenté est : minimum 5 lettres (majuscules ou minuscules) et minimum 2 chiffres. J'ai ainsi utilisé des regexes afin de tester les critères, voici ce qui se passe dans le frontend :

1. L'utilisateur entre un caractère dans le champ du mot de passe
 2. Un événement listener se déclenche et appelle la méthode de test du mot de passe
 3. La méthode de test du mot de passe essaye de match différentes Regexes afin de déterminer la force du mdp.
 4. La barre de progression change de taille et de couleur, les critères remplis deviennent vert.
 5. L'utilisateur est satisfait du mot de passe et valide le formulaire.
 6. La fonction de submission du formulaire vérifie que le mot de passe soit conforme aux critères.
- Si oui, on continue normalement
 - Si non, une 'alert' est provoquée

Dans le cas où l'utilisateur bypass le frontend avec un mot de passe non-conforme, nous ajoutons une vérification dans le Backend que voici. Dans AuthController.js - partie register du controlleur. Nous ajoutons donc :

```
//test de force du mot de passe

const level1 = /[a-zA-Z]{5,}/; //5 lettres (minuscule ou/et majuscule)
const level2 = /[0-9]{2,}/; //2 chiffres
if (!level1.test(password) || !level2.test(password)) {
  return res.status(400).json({
    error:
      "Votre mot de passe doit comporter au moins 5 lettres et 2 chiffres.",
  });
}
```

11 Limiter la durée du token jwt

À présent, le but est de rendre le token plus safe pour l'utilisateur. En effet, ce token permet de faire pas mal de choses et donc il est important de limiter ce token.

Nous allons donc ajouter un délais d'expiration au token : 15 minutes. `{ expiresIn: "15m" }` Après ces 15 minutes le token ne sera plus valide et l'utilisateur devra être revérifié.

Enfin ça ce serait embêtant, devoir entrer username - mot de passe tous les quart d'heures. C'est pourquoi nous implémentons aussi un refresh token qui permet de générer un nouvel access token.

Nous créons donc une nouvelle route afin de pouvoir créer un nouveau token

```
router.post("/refresh", verifyRefreshToken, controller.refreshToken);
```

verifyRefreshToken est une nouvelle méthode qui vient du middleware et qui, comme son nom l'indique, va vérifier le token de rafraîchissement. controller.refreshToken est aussi une nouvelle méthode du controlleur d'authentification qui va créer et retourner un nouvel access token.

Nous utilisons cette méthode dans le middleware, uniquement si ce dernier renvoie une erreur de type *TokenExpiredError*.

12 Audit des dépendances npm

Il devient important de réaliser un "audit npm". En effet, npm choisit parfois des packages vulnérables et introduit donc des failles dans l'application. Nous lançons donc la commande `npm audit` et voici ce que j'obtient :

Nom	Degré	Utilisation	Souci	Conditions
Body-parser	haut	Converti le body des requêtes	Dénial of Service	l'encodage des url est activé
Brace-expansion	modéré	Génère string en se basant sur des regex	Boucle infinie, consommation de temps-ressources-mémoire	Certains patternes
Braces	haut	Calcul des résultats d'une pseudo-regex	Pareil, mauvaise gestion des ressources	Certains patternes

Nom	Degré	Utilisation	Souci	Conditions
Cookie	bas	Création de cookies	Accepte des cookies avec des caractères indésirables	-
Minimatch	haut	Patternes de Regex pour fichiers (*.js)	Prend beaucoup plus de temps que nécessaire	Certains patternes
Openssl	critique	Execution de commandes	Utilisation afin de lancer des commandes malveillantes	-
path-to-regexp	haut	Détection d'url à partir de Regex	Similaire à Minimatch	Certains patternes
Picomatch	haut	Regex avancées	Mauvaise interprétation de classes de caractères	Certains patternes
qs	modéré	Transformer les Objets JS en query db	Consommation excessive de mémoire	Indentation de tableaux profonde
send	bas	Partages de données et de fichiers	Vulnérable aux injections html, failles XSS	Injections

Maintenant que nous savons tout ça, nous pouvons demander à npm de tout réparer : lancez `npm audit fix`. Et que la magie opère.

14 Gérer les exceptions

Nous devons faire en sorte que le serveur ne donne pas trop d'infos au client. En effet, si le client envoie une mauvaise requête ou une requête malveillante, le serveur ne doit pas envoyer tous les détails de l'erreur. Cela permettrait à un potentiel attaquant de trouver des failles.

Nous allons donc checker tous les retours JSON, console.log et nous allons en profiter pour ajuster les status codes http. J'ai eu des changements à faire dans les contrôleurs, le fichier `server.js`, ainsi que certaines vues.

15 limiter le nombre de tentatives de login

Une tâche simple et rapide : Afin de parer au brute-force, nous devons limiter le nombre de tentatives de login par ip. Nous allons mettre la limite à 5 tentative par IP par minute. L'exemple de la doc (<https://www.npmjs.com/package/express-rate-limit>) est parfaitement suffisant. Nous l'ajoutons donc au fichier `server.js`, mais nous adaptons la route qui devient :

```
app.post("/api/auth/login", limiter);
```

Et c'est déjà terminé, merci express-rate-limit !!

Conclusion

Générale

En conclusion, ce projet aura été globalement réussi. En effet, un total de 15 points aura été validé. Les tâches ci-dessous ont été réalisées :

- Rendre le login fonctionnel
- Rendre le register fonctionnel
- Hasher les mots de passes en db
- Ajouter le sel
- Ajouter le poivre
- Corriger les requêtes SQL afin de parer l'injection
- Implémenter un token jwt
- Implémenter les rôles via jwt et protéger les routes
- Implémenter le *https*
- Ajouter la politique de mot de passe
- Limiter la durée du token jwt
- Réaliser un audit des dépendances *npm*
- Gérer les exceptions
- Limiter le nombre de tentatives de login par *ip* par minute

Personnelle

Personnellement, j'ai bien aimé ce projet. Ceci étant je trouve que nous aurions mérité d'avoir de la théorie sur *Express* en premier temps (dans un autre module). En effet, j'ai mis beaucoup trop de temps à essayer de comprendre comment l'application fonctionne et à quoi sert quoi.

De même, j'aurais bien aimé avoir un peu plus de théorie, mais surtout d'exercices pratique durant ce module à propos des tâches à réaliser. Par exemple, avoir un exercice pratique sur le protocole *https* m'aurait énormément aidé à savoir comment ça fonctionne et par où commencer pour l'implémenter.